

Source Asia - Backend Assignment

Thank you for applying to Source Asia.

As a next step, please complete the backend assignment below. It has **two required parts**. Submit everything in **one GitHub repository** (or a zip if GitHub is not possible).

General instructions

Item	Detail
Deadline	Saturday, 5:00 PM. Reply to this email with your submission link
Language	Go preferred. Node.js is acceptable
Storage	In-memory is fine for both parts (maps, slices, etc.)
External data	No dataset is provided. You create all data yourself via your API (see Part 2)
AI tools	Allowed; mention in README if you used them and for what
Scope	Build a runnable HTTP service . CLI-only solutions are not accepted

Part 1 - Rate-limited API (required)

Build HTTP endpoints on the same service (or a separate binary in the same repo; your choice; document how to run each part).

Endpoints

****POST /request****

- **Request body (JSON):**

```
{
  "user_id": "string (required, non-empty)",
  "payload": "any JSON value (required)"
}
```

- **Success:** 201 Created or 200 OK. Document your choice in the README
- **Body:** confirm the request was accepted (your shape; document it)

****GET /stats****

- Returns statistics **per user** (and optionally global totals; your choice)

- Must include at least:
 - number of **accepted** requests in the **current** 1-minute window (per user)
 - number of **rejected** requests due to rate limiting (per user; document whether cumulative or per window)
- Response format: JSON. Document the schema in the README

Rate limiting rules

- Maximum **5 accepted requests per user_id** per rolling or fixed 1-minute window. Document which approach you used
- When the limit is exceeded: return ****429 Too Many Requests**** (or another 4xx if you justify it in the README)
- Response body must include a clear error message (JSON recommended)

Technical requirements

- Store data **in memory** (no database required for Part 1)
- Must be **correct under concurrent requests** (parallel calls for the same **user_id** must not allow more than 5 accepts per minute)
- Invalid input (**missing user_id**, empty **user_id**, invalid JSON): return ****400 Bad Request****
- README must briefly describe **production limitations** (e.g. single instance, restart loses state, multi-instance deployment)

How to test (your responsibility)

- Provide README steps to run the server and example **curl** commands (or equivalent)
- You do not need to submit automated tests, but they are a plus

Part 2 - Product catalog with media (required)

Extend the **same repository** with product APIs (same process or a second port; document clearly).

Concept

- A **product** has core fields and **many images and many videos**
- Media are **URL strings only** (e.g. **https://cdn.example.com/path/file.jpg**)
- **Do not** accept file uploads, binary bodies, or base64 image data in the API
- **Do not** integrate with a real CDN or external catalog. We evaluate **your API design**, not hosting

Data: where it comes from

- No seed file or external API is provided
- All products and media URLs are created **through your endpoints** using example URLs you invent, for example:

```
{
  "name": "Widget A",
  "sku": "SKU-001",
  "image_urls": [
    "https://cdn.example.com/products/sku-001/img-1.jpg",
    "https://cdn.example.com/products/sku-001/img-2.jpg"
  ],
  "video_urls": [
    "https://cdn.example.com/products/sku-001/demo.mp4"
  ]
}
```

- For your own testing, you may add an optional **seed script** or documented loop (e.g. “run this to create 1,000 products”). **Not required for submission**, but useful to prove GET /products stays fast

Endpoints

****POST /products****

- Request body (JSON):

```
{
  "name": "string (required, non-empty)",
  "sku": "string (required, non-empty, unique)",
  "image_urls": ["optional array of URL strings"],
  "video_urls": ["optional array of URL strings"]
}
```

- Success: 201 Created with the created product (include id you assign)
- Duplicate sku: 409 Conflict (or 400; document choice)
- Validation failures: 400 Bad Request (see Validation below)

****GET /products****

- **Purpose:** list page / grid for a UI. Must stay **fast** when there are many products, each with many media URLs stored in the system
- **Must not** return the full `image_urls` and `video_urls` arrays for every product in the list response
- Allowed in list items: e.g. `id`, `name`, `sku`, `image_count`, `video_count`, `thumbnail_url` (optional), `created_at` (optional). Your design; document it

- **Pagination required:** support `limit` and `offset` (or `page` and `page_size`) query parameters; document defaults and max limits

****GET /products/{id}****

- **Purpose:** detail page
- Returns the full product including **all** `image_urls` and `video_urls`
- Unknown id: 404 Not Found

****POST /products/{id}/media****

- **Request body (JSON):**

```
{
  "image_urls": ["optional, at least one of image_urls or video_urls required"],
  "video_urls": ["optional"]
}
```

- Appends new URLs to the existing product
- Unknown id: 404 Not Found
- Empty body (no URLs): 400 Bad Request

Validation (required)

- Reject empty `name` or `sku`
- Reject invalid URLs (must be `http://` or `https://`, reasonable max length e.g. 2048 chars; document your rules)
- Enforce a **maximum number of URLs per request** (e.g. max 20 per array). Document the limit
- Reject duplicate `sku` on create

Performance rule (important)

- With **1,000 products** and **10 image URLs each** stored in your system, `GET /products?limit=20` must not load or serialize all 10,000 image URLs. Only return what the list endpoint needs
- Explain in the README: **data model** (how you store products and media in memory), **how list vs detail queries differ**, and what you would change with **PostgreSQL + a CDN** in production

Storage

- In-memory storage is sufficient (nested maps, separate media structure, etc.)
- Persistence across restarts is **not** required

Submission

Reply to **this email** by **Saturday, 5:00 PM** with:

1. **GitHub repository link**
2. **README** (required)
3. **Honest note** if any requirement is incomplete (what's missing and why)

We will review: correctness, concurrency safety (Part 1), API shape and list-vs-detail design (Part 2), code clarity, and README quality.

If anything in this brief is unclear, reply to this email **before** the deadline with your question.

Good luck.